

Neural Network Training Process

Moin Mostakim

April 26, 2025

Neural Network Training Process with Batch Normalization

Batch Creation, Epochs, and Batch Normalization Explained with Matrices

- We will walk through the training process of a simple feedforward neural network, focusing on:
 - Batch creation
 - Epochs
 - Batch normalization (BN)
- All steps will use matrix representations for clarity.

Step 1: Dataset and Batch Creation

- **Dataset:** Assume we have a dataset with $N = 1000$ samples, each with $d = 4$ features, and a target variable (e.g., binary classification).
- Input matrix $X \in \mathbb{R}^{N \times d}$, target vector $Y \in \mathbb{R}^{N \times 1}$.

$$X = \begin{bmatrix} x_{1,1} & x_{1,2} & x_{1,3} & x_{1,4} \\ x_{2,1} & x_{2,2} & x_{2,3} & x_{2,4} \\ \vdots & \vdots & \vdots & \vdots \\ x_{1000,1} & x_{1000,2} & x_{1000,3} & x_{1000,4} \end{bmatrix}, \quad Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{1000} \end{bmatrix}$$

- **Batch Creation:** Divide the dataset into mini-batches of size $B = 32$.
- Number of batches per epoch: $\frac{N}{B} = \frac{1000}{32} \approx 31.25$, so we have 31 full batches, and the last batch has 8 samples.
- A batch $X_b \in \mathbb{R}^{B \times d}$, $Y_b \in \mathbb{R}^{B \times 1}$, e.g., first batch:

$$X_b = \begin{bmatrix} x_{1,1} & x_{1,2} & x_{1,3} & x_{1,4} \\ \vdots & \vdots & \vdots & \vdots \\ x_{32,1} & x_{32,2} & x_{32,3} & x_{32,4} \end{bmatrix}, \quad Y_b = \begin{bmatrix} y_1 \\ \vdots \\ y_{32} \end{bmatrix}$$

Step 2: Epochs

- **Epoch:** One epoch is a complete pass through the entire dataset.
- With $N = 1000$, $B = 32$, one epoch consists of processing 31 batches of 32 samples and 1 batch of 8 samples.
- If we train for $E = 10$ epochs, the network will process the entire dataset 10 times, i.e., $10 \times 32 = 320$ batches total.
- **Shuffling:** At the start of each epoch, shuffle the dataset to ensure the model doesn't learn the order of the data:

$$\text{Shuffled } X = \begin{bmatrix} x_{\pi(1),1} & x_{\pi(1),2} & x_{\pi(1),3} & x_{\pi(1),4} \\ \vdots & \vdots & \vdots & \vdots \\ x_{\pi(1000),1} & x_{\pi(1000),2} & x_{\pi(1000),3} & x_{\pi(1000),4} \end{bmatrix}$$

where π is a random permutation of indices $\{1, 2, \dots, 1000\}$.

Step 3: Forward Pass with Batch Normalization (Part 1)

Network Architecture and Linear Transformation

- **Network Architecture:** A simple 2-layer network:
 - Input layer: 4 neurons (features)
 - Hidden layer: 8 neurons (with ReLU activation)
 - Output layer: 1 neuron (sigmoid for binary classification)
- **Weights and Biases:**

$$W_1 \in \mathbb{R}^{4 \times 8}, \quad b_1 \in \mathbb{R}^{1 \times 8}, \quad W_2 \in \mathbb{R}^{8 \times 1}, \quad b_2 \in \mathbb{R}^{1 \times 1}$$

- For a batch $X_b \in \mathbb{R}^{32 \times 4}$:

$$Z_1 = X_b W_1 + b_1 \quad (\text{Linear transformation, } Z_1 \in \mathbb{R}^{32 \times 8})$$

Step 3: Forward Pass with Batch Normalization (Part 2)

Batch Normalization Process

- **Batch Normalization (BN):** Applied to Z_1 before activation.

- 1 Compute the mean of each feature (column) in the batch:

$$\mu_j = \frac{1}{B} \sum_{i=1}^B Z_{1,i,j}, \quad \mu \in \mathbb{R}^{1 \times 8}$$

- 2 Compute the variance:

$$\sigma_j^2 = \frac{1}{B} \sum_{i=1}^B (Z_{1,i,j} - \mu_j)^2, \quad \sigma^2 \in \mathbb{R}^{1 \times 8}$$

- 3 Normalize:

$$\hat{Z}_{1,i,j} = \frac{Z_{1,i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}, \quad \epsilon = 10^{-5} \text{ (for numerical stability)}$$

- 4 Scale and shift (learnable parameters $\gamma, \beta \in \mathbb{R}^{1 \times 8}$):

$$BN(Z_1)_{i,j} = \gamma_j \hat{Z}_{1,i,j} + \beta_j$$

Step 3: Forward Pass (Continued)

- **Activation:** Apply ReLU to the batch-normalized output:

$$H_1 = \text{ReLU}(\text{BN}(Z_1)), \quad H_1 \in \mathbb{R}^{32 \times 8}$$

- Second layer:

$$Z_2 = H_1 W_2 + b_2, \quad Z_2 \in \mathbb{R}^{32 \times 1}$$

- Apply sigmoid for binary classification:

$$\hat{Y}_b = \sigma(Z_2) = \frac{1}{1 + e^{-Z_2}}, \quad \hat{Y}_b \in \mathbb{R}^{32 \times 1}$$

- **Loss:** Compute binary cross-entropy loss for the batch:

$$L = -\frac{1}{B} \sum_{i=1}^B \left[Y_{b,i} \log(\hat{Y}_{b,i}) + (1 - Y_{b,i}) \log(1 - \hat{Y}_{b,i}) \right]$$

Step 4: Backward Pass (Backpropagation)

- Compute gradients of the loss L with respect to parameters:

- Gradient w.r.t. Z_2 :

$$\frac{\partial L}{\partial Z_2} = \hat{Y}_b - Y_b \quad (\text{due to sigmoid and binary cross-entropy})$$

- Gradient w.r.t. W_2, b_2 :

$$\frac{\partial L}{\partial W_2} = H_1^T \frac{\partial L}{\partial Z_2}, \quad \frac{\partial L}{\partial b_2} = \sum_{i=1}^B \frac{\partial L}{\partial Z_{2,i}}$$

- Backprop through H_1 , BN, and ReLU to compute $\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial b_1}, \frac{\partial L}{\partial \gamma}, \frac{\partial L}{\partial \beta}$.

- **Update Parameters:** Use gradient descent with learning rate $\eta = 0.01$:

$$W_2 \leftarrow W_2 - \eta \frac{\partial L}{\partial W_2}, \quad b_2 \leftarrow b_2 - \eta \frac{\partial L}{\partial b_2}$$

$$W_1 \leftarrow W_1 - \eta \frac{\partial L}{\partial W_1}, \quad b_1 \leftarrow b_1 - \eta \frac{\partial L}{\partial b_1}$$

$$\gamma \leftarrow \gamma - \eta \frac{\partial L}{\partial \gamma}, \quad \beta \leftarrow \beta - \eta \frac{\partial L}{\partial \beta}$$

Step 5: Iterate Over Batches and Epochs

- **For each epoch $e = 1$ to E :**

- ① Shuffle the dataset.
- ② Split into batches.
- ③ For each batch:
 - Forward pass (with BN).
 - Compute loss.
 - Backward pass.
 - Update parameters.

- **Batch Normalization During Inference:**

- Use running averages of μ and σ^2 (computed during training) to normalize test data.
- Apply the same γ and β .

Summary of Key Points

- **Batch Creation:** Splits data into manageable chunks (e.g., 32 samples per batch).
- **Epochs:** Number of full passes through the dataset (e.g., 10 epochs).
- **Batch Normalization:**
 - Normalizes activations within each batch.
 - Improves training stability and speed.
 - Introduces learnable parameters γ, β .
- **Matrix Operations:** All steps (forward, backward, BN) are expressed as matrix operations for efficiency.